



ATELIER POO EN JAVA

De la procédurale à l'objet

Dans l'atelier précédent, on a construit la facture en **procédural** : les données d'un côté (des tableaux de prix, de quantités), les traitements de l'autre (des méthodes static comme totalHT). Ça fonctionnait, mais souviens-toi du code des « 3 produits » :

```
String[] noms = {"Stylo", "Cahier", "Gomme"};
double[] prix = {20.0, 15.0, 8.0};
int[] qte = {3, 2, 5};
```

Le problème : **trois tableaux à garder synchronisés**. Le prix du produit *i* est dans `prix[i]`, sa quantité dans `qte[i]`... Si on trie l'un, tout se casse. Et le calcul `prix[i] * qte[i]` est « à part », il ne fait pas partie du produit.

NOTION — L'idée de la POO

La **programmation orientée objet** regroupe dans un même **objet** les *données* d'un produit (nom, prix, quantité) et les *actions* qui les concernent (calculer le sous-total). Un produit devient une chose cohérente, pas trois cases dispersées.

Progression de l'atelier

#	Concept POO	Ce qu'on construit
1	Classe & objet	le moule Produit et des exemplaires
2	Constructeur & this	créer un produit en une ligne
3	Méthodes d'instance	le produit sait calculer son sous-total
4	Encapsulation	protéger prix et quantité (private)
5	Énumérations (enum)	un statut sûr et fermé pour le produit
6	Tableau d'objets	un panier de produits
7	Collections (ArrayList)	un panier qui grandit sans limite de taille
8	Composition	une Facture qui contient des Produits
9	toString()	afficher une ligne de facture lisible
10	Héritage	produits alimentaire / électronique
11	Polymorphisme	une TVA par type, une seule boucle
12	L'abstraction (classe abstraite)	Produit devient un modèle qu'on ne peut pas instancier

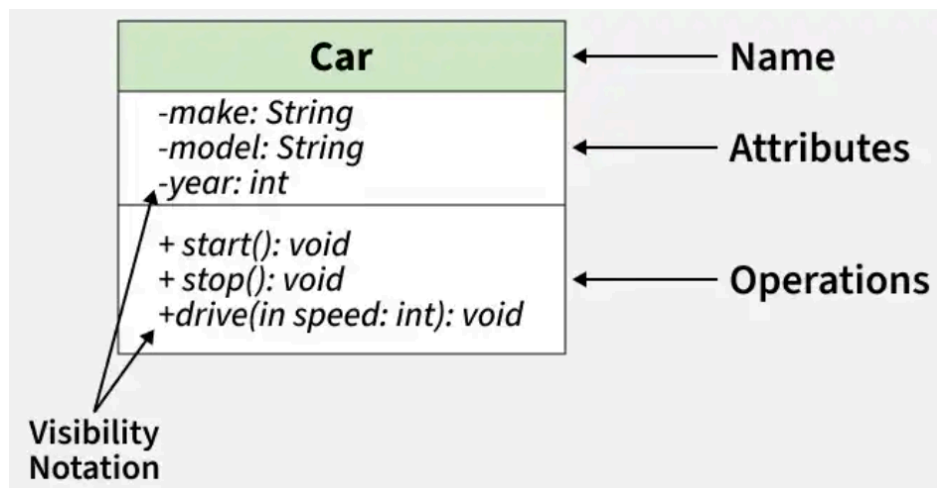
Challenge 1 — Classe et objet

OBJECTIF DU CHALLENGE

Crée une **classe** Produit décrivant ce qu'est un produit (nom, prix, quantité). Puis, dans main, crée **deux objets** Produit et affiche leur nom.

NOTION — Classe vs objet

Une **classe** est un *moule* : elle décrit la forme (les attributs). Un **objet** est un *exemplaire* fabriqué avec **new**. D'une classe Produit, on crée autant de produits qu'on veut, chacun avec ses propres valeurs.



NOTION — Les attributs

nom, prix, quantité sont les **attributs** de la classe : les données que chaque objet transporte. On y accède avec le point : `p1.prix`.

À TON TOUR

Ajoute un attribut `String code` (référence produit) à la classe, crée un 3e produit « Gomme » et affiche son code.

Challenge 2 — Le constructeur et this

OBJECTIF DU CHALLENGE

Ajoute un **constructeur** à Produit pour créer un produit **complet en une seule ligne**.

NOTION — Le constructeur

Un **constructeur** est une méthode spéciale appelée par new. Il porte **le nom exact de la classe** et **n'a pas de type de retour**. Son rôle : initialiser l'objet à sa création.

NOTION — Le mot-clé this

this désigne « l'objet en cours de création ». `this.prix = prix` range le paramètre `prix` dans l'attribut `prix` de cet objet. `this` lève l'ambiguïté quand paramètre et attribut portent le même nom.

À TON TOUR

Ajoute le code produit au constructeur : `Produit(String code, String nom, double prix, int quantite)`. Crée deux produits.

Challenge 3 — Les méthodes d'instance

OBJECTIF DU CHALLENGE

Le calcul du sous-total doit **appartenir au produit**. Ajoute une méthode `sousTotal()` qui retourne `prix × quantité`, et `appliquerRemise(double taux)` qui baisse le prix.

NOTION — Méthode d'instance vs static

Une **méthode d'instance** (`p.sousTotal()`) agit sur **un objet précis** et accède directement à ses attributs — plus besoin de lui passer `prix` et `quantite` en paramètres, ils sont *dans* l'objet ! C'est toute la différence avec l'ancienne `totalHT(prix, qte)` en static.

À TON TOUR

Ajoute une méthode `double ttc(double tauxTVA)` qui retourne le sous-total TVA comprise. Teste-la avec 20 %.

Challenge 4 — L'encapsulation

OBJECTIF DU CHALLENGE

Rien n'empêche d'écrire `p.prix = -50` ou une quantité négative. **Protège** les attributs : rends-les `private` et donne un accès contrôlé via des **getters/setters** avec validation.

NOTION — Encapsulation

private rend un attribut **inaccessible depuis l'extérieur**. On l'expose via des méthodes public : un **getter** pour lire, un **setter** pour écrire. L'avantage décisif : le setter peut *valider* la donnée — ici, refuser un prix négatif. L'objet protège sa propre cohérence.

Access Modifiers

Modifieur	Class	Package	SubClass	Global
Public	✓	✓	✓	✓
Protected	✓	✓	✓	✗
Default	✓	✓	✗	✗
Private	✓	✗	✗	✗

NOTION — Convention getter/setter

Pour un attribut `prix` : `getPrix()` pour lire, `setPrix(...)` pour écrire. C'est une convention universelle en Java (pour un boolean, le getter devient `isXxx()`).

À TON TOUR

Ajoute le getter `getNom()` et vérifie qu'un `setPrix(-10)` ne change pas le prix (il reste l'ancien).

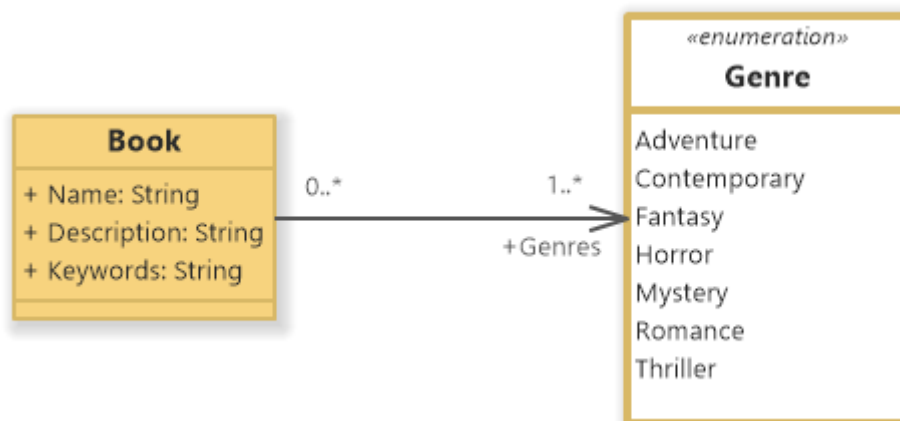
Challenge 5 — Les énumérations (enum)

OBJECTIF DU CHALLENGE

Un produit a un statut : disponible, en rupture, ou retiré de la vente. Si on le stocke dans une simple String, rien n'empêche d'écrire "disponible", "DISPO" ou "en stock" — autant de valeurs incohérentes qui casseront les comparaisons. Crée un enum Statut qui limite les valeurs possibles à une liste fixe et sûre, et ajoute-le à la classe Produit.

NOTION — Qu'est-ce qu'un enum

Un enum (énumération) est un type spécial dont les valeurs sont limitées à un ensemble fixe de constantes que tu définis toi-même (DISPONIBLE, RUPTURE, RETIRE). C'est parfait pour représenter une donnée qui ne peut prendre qu'un nombre connu et limité de valeurs : un statut, un jour de la semaine, un niveau de priorité, une catégorie.



NOTION — Pourquoi c'est mieux qu'une String

Trois avantages décisifs sur la String :

- Sécurité à la compilation : `Statut.DISPONIBL` (faute de frappe) provoque une erreur de compilation immédiate, alors que "disponibl" passerait inaperçu jusqu'au bug.
- Valeurs limitées : il est impossible d'affecter une valeur hors de la liste. Le type est « fermé ».

- Lisibilité : le code documente lui-même les seuls états possibles d'un produit.

NOTION — Comparer des enums

Contrairement aux String (où il fallait `equals()`), on compare les enums avec `==` sans risque : `statut == Statut.RUPTURE`. Chaque constante d'enum est un objet unique, donc `==` fonctionne parfaitement et reste la façon idiomatique de les comparer.

À TON TOUR

Ajoute une valeur PROMOTION à l'enum Statut. Dans la méthode `message()`, ajoute le case correspondant qui affiche "nom est en promotion". Crée un produit, passe-le en PROMOTION et vérifie l'affichage. Bonus : ajoute une méthode `int nombreDeStatuts()` qui retourne `Statut.values().length` (le nombre total de statuts définis).

Challenge 6 — Un tableau d'objets

OBJECTIF DU CHALLENGE

Fini les tableaux parallèles ! Range plusieurs Produit dans **un seul tableau** (le panier), parcours-le et calcule le **total HT**.

NOTION — Une collection cohérente

Un **tableau de Produit** remplace les trois tableaux parallèles du départ. Chaque case contient un objet complet (nom + prix + quantité) : impossible de les désynchroniser. **C'est le problème initial, résolu.** Et la boucle est plus lisible : `p.sousTotal()` au lieu de `prix[i] * qte[i]`.

À TON TOUR

Applique une remise de 10 % à tous les produits du panier (boucle + `appliquerRemise`), puis recalcule le total HT.

Challenge 7 — Du tableau à la collection (ArrayList)

OBJECTIF DU CHALLENGE

Au challenge précédent, on a rangé les produits dans un tableau `Produit[]`. Mais un tableau a une taille fixe : impossible d'ajouter un produit une fois qu'il est plein. On va d'abord provoquer volontairement cette limite, puis la dépasser avec une collection dynamique, l'`ArrayList`, qui grandit toute seule.

Tableau vs ArrayList

Deux façons de stocker plusieurs objets en Java

TABLEAU

```
Produit[] panier = new Produit[3];
```

Stylo

Cahier

Gomme

[0]

[1]

[2]

[3]

Taille fixe : impossible d'ajouter la 4e case

ARRAYLIST

```
ArrayList<Produit> panier = ...
```

Stylo

Cahier

Gomme

Règle

+

Taille dynamique : `add()` autant qu'on veut

Critère	Tableau	ArrayList
Taille	fixe (à la création)	dynamique (grandit seule)
Ajouter	impossible si plein	<code>add()</code>
Nombre d'éléments	<code>.length</code>	<code>.size()</code>
Accès élément	<code>panier[0]</code>	<code>panier.get(0)</code>
Supprimer	à la main (recopie)	<code>remove()</code>

Taille connue et figée → tableau. Nombre d'éléments qui varie → ArrayList.

NOTION — Le type générique <Produit>

Les chevrons `<Produit>` indiquent le type des éléments que la liste contiendra. `ArrayList<Produit>` est « une liste de produits ». Ça garantit qu'on ne peut y mettre que des `Produit` : `panier.add("texte")` provoquerait une erreur de compilation. C'est la généricité : la même classe `ArrayList` s'adapte à n'importe quel type d'objet (`ArrayList<String>`, `ArrayList<Produit>`...).

NOTION — L'ArrayList ne stocke que des objets

Une **ArrayList** ne peut contenir que des objets, pas des types primitifs. Pour des nombres, on utilise les classes wrapper : **ArrayList<Integer>** (pas `int`),

ArrayList<Double> (pas **double**). Java convertit automatiquement dans la plupart des cas (**liste.add(5)** fonctionne pour une **ArrayList<Integer>**).

NOTION — **.size()** et non **.length**

Confusion classique : un **tableau** utilise **.length** (sans parenthèses, une propriété), une **ArrayList** utilise **.size()** (avec parenthèses, une méthode). Et l'accès se fait par **.get(i)**, pas par les crochets **[i]**.

À TON TOUR

Pars du panier en **ArrayList** ci-dessus :

1. Ajoute une méthode qui **supprime** le produit à l'indice 0, puis réaffiche le total.
2. Ajoute 5 produits dans une boucle **for** et vérifie que la liste s'agrandit sans erreur.
3. **Bonus** : ne compte le total que des produits **achetables** (réutilise **estAchetable()** de l'enum **Statut**, si tu l'as ajouté).

Challenge 8 — La composition

OBJECTIF DU CHALLENGE

Crée une classe Facture qui **contient** un tableau de Produit et sait **ajouter un produit, calculer le total HT** et le **total TTC**. La logique quitte main pour vivre dans l'objet Facture.

NOTION — La composition

Un objet Facture **est composé d'**objets Produit (relation « a des »). Chaque classe a **sa responsabilité** : Produit gère un article, Facture gère la collection et les totaux. C'est le cœur d'une bonne conception objet.

À TON TOUR

Ajoute une méthode calculerRemise() à Facture (selon le total HT, comme dans l'atelier procédural) et intègre-la au calcul du TTC.

Challenge 9 — La méthode toString()

OBJECTIF DU CHALLENGE

Afficher un produit avec `System.out.println(produit)` donne `Produit@1b6d`.
Redéfinis `toString()` pour obtenir une **ligne de facture** lisible et formatée.

NOTION — Redéfinir toString()

Toute classe hérite d'une méthode `toString()` (de la classe `Object`). Par défaut elle renvoie l'adresse mémoire. En la **redéfinissant**, on décide de la représentation texte. Java l'appelle **automatiquement** dans `println` et les concaténations. L'annotation `@Override` signale qu'on redéfinit une méthode existante.

À TON TOUR

Ajoute un `toString()` à la classe `Facture` qui affiche toutes ses lignes puis le total TTC.

Challenge 10 — L'héritage

OBJECTIF DU CHALLENGE

Tous les produits n'ont pas la même TVA : les produits **alimentaires** sont à 7 %, les produits **standard** à 20 %. Crée une classe de base `Produit`, puis `ProduitAlimentaire` et `ProduitStandard` qui en **héritent**, chacun avec son taux.

NOTION — Héritage : `extends` et `super`

`extends` crée une relation « est un » : un `ProduitAlimentaire` *est un* `Produit`. La classe fille **hérite** des attributs et méthodes de la mère, et peut en **redéfinir** (ici `tauxTVA()`). **`super(...)`** appelle le constructeur de la classe mère.

NOTION — `protected`

`protected` est entre `private` et `public` : l'attribut est accessible dans la classe **et dans ses classes filles**. Pratique pour que les filles réutilisent `prix` et `quantite`.

À TON TOUR

Crée une classe `ProduitElectronique` avec TVA 20 % et un attribut `garantie` (en mois) affiché dans son `toString()`.

Challenge 11 — Le polymorphisme

OBJECTIF DU CHALLENGE

Range des produits **alimentaires et standard** dans **un même tableau de Produit**, et calcule le total TTC en une seule boucle. Chaque produit appliquera **sa propre TVA**, automatiquement.

NOTION — Le polymorphisme

« Poly-morphe » = « plusieurs formes ». Le tableau contient des `ProduitAlimentaire` et des `ProduitStandard`, mais l'appel `p.ttc()` exécute automatiquement le `tauxTVA()` **propre au type réel** de chaque objet (7 % ou 20 %). On écrit **un seul code générique** qui s'adapte tout seul : c'est la puissance de la POO.

NOTION — Pourquoi c'est puissant

Pour ajouter un nouveau type (un `ProduitDeLuxe` à 30 % par exemple), il suffit de créer la classe qui hérite de `Produit`. La boucle de calcul **ne change pas** : elle traite déjà « tout `Produit` ». Le code existant reste intact.

À TON TOUR

Ajoute ta classe `ProduitElectronique` au panier et vérifie que le total TTC se met à jour correctement, sans toucher à la boucle.

Challenge 12 — L'abstraction (classe abstraite)

OBJECTIF DU CHALLENGE

Au challenge précédent, **Produit** était la classe mère de **ProduitAlimentaire** et **ProduitStandard**. Mais une question se pose : **un Produit « générique » a-t-il un sens tout seul ?** Un produit sans catégorie n'existe pas dans la vraie vie — il est *forcément* alimentaire, standard, électronique... Rends la classe **Produit abstraite** : un modèle qu'on ne peut pas instancier directement, et qui **impose** à ses filles de définir leur propre taux de TVA.

NOTION — La classe abstraite

Une **méthode abstraite** est déclarée **sans corps** (**public abstract double tauxTVA();**), suivie d'un simple point-virgule. Elle définit *ce qu'il faut faire* sans dire *comment*. Toute classe fille **est obligée** de la définir (**@Override**), sinon elle ne compile pas. C'est une **garantie** : impossible d'oublier de préciser la TVA d'une nouvelle catégorie de produit — le compilateur l'exige.

NOTION — Pourquoi c'est puissant

Factoriser le commun : le calcul du TTC (**sousTotal() + sousTotal() * tauxTVA()**) est écrit **une seule fois** dans **Produit**, hérité par toutes les filles.

Imposer le spécifique : chaque fille **doit** fournir son **tauxTVA()**. Le « quoi » est garanti par la classe mère, le « comment » est délégué aux filles.

C'est le principe « **programmer un concept, pas une chose concrète** » : **Produit** décrit l'idée de produit, les sous-classes lui donnent une réalité.

À TON TOUR

Crée une classe **ProduitExonere** (TVA à 0 %, pour les produits de première nécessité comme les médicaments) qui hérite de **Produit** — elle **doit** définir **tauxTVA()**.

Vérifie qu'écrire **new Produit(...)** provoque bien une **erreur de compilation**.

Bonus : tente de créer une classe fille **sans** redéfinir **tauxTVA()** et observe l'erreur du compilateur — c'est la preuve que l'abstraction *force* le contrat.